

PL/SQL Practical Solutions

Questions 19 – 60 | Oracle PL/SQL Programs, Cursors, Procedures, Packages & Triggers

(19) Generate a PL/SQL block to find Odd or Even number.

```
DECLARE
    n NUMBER := &n;
BEGIN
    IF MOD(n,2) = 0 THEN
        DBMS_OUTPUT.PUT_LINE(n || ' is Even');
    ELSE
        DBMS_OUTPUT.PUT_LINE(n || ' is Odd');
    END IF;
END;
/
```

(20) Generate a PL/SQL block to enter any number and print whole table of the entered number.

```
DECLARE
    n NUMBER := &n;
    i NUMBER := 1;
BEGIN
    WHILE i <= 10 LOOP
        DBMS_OUTPUT.PUT_LINE(n || ' x ' || i || ' = ' || (n*i));
        i := i + 1;
    END LOOP;
END;
/
```

(21) Write a PL/SQL program to find entered year is Leap year or not.

```
DECLARE
    yr NUMBER := &yr;
BEGIN
    IF (MOD(yr,4) = 0 AND MOD(yr,100) != 0) OR MOD(yr,400) = 0 THEN
        DBMS_OUTPUT.PUT_LINE(yr || ' is a Leap Year');
    ELSE
        DBMS_OUTPUT.PUT_LINE(yr || ' is not a Leap Year');
    END IF;
END;
/
```

(22) Generate a PL/SQL block to print 1 to 5 number using while loop.

```
DECLARE
    i NUMBER := 1;
BEGIN
    WHILE i <= 5 LOOP
        DBMS_OUTPUT.PUT_LINE(i);
        i := i + 1;
    END LOOP;
END;
/
```

(23) Generate a PL/SQL block to print 1 to 10 number using simple loop.

```
DECLARE
    i NUMBER := 1;
```

```

BEGIN
  LOOP
    DBMS_OUTPUT.PUT_LINE(i);
    EXIT WHEN i = 10;
    i := i + 1;
  END LOOP;
END;
/

```

(24) Generate a PL/SQL block to print 1 to 10 number using for loop.

```

BEGIN
  FOR i IN 1..10 LOOP
    DBMS_OUTPUT.PUT_LINE(i);
  END LOOP;
END;
/

```

(25) Generate a PL/SQL block to enter rollno, name, mark1, mark2, mark3 and find out the total, percentage and appropriate class.

```

DECLARE
  rollno NUMBER      := &rollno;
  sname  VARCHAR2(30) := '&sname';
  m1 NUMBER := &m1;
  m2 NUMBER := &m2;
  m3 NUMBER := &m3;
  total NUMBER;
  perc  NUMBER;
  class VARCHAR2(20);
BEGIN
  total := m1 + m2 + m3;
  perc  := total / 3;

  IF perc >= 75 THEN
    class := 'Distinction';
  ELSIF perc >= 60 THEN
    class := 'First Class';
  ELSIF perc >= 50 THEN
    class := 'Second Class';
  ELSIF perc >= 40 THEN
    class := 'Pass Class';
  ELSE
    class := 'Fail';
  END IF;

  DBMS_OUTPUT.PUT_LINE('Roll No : ' || rollno);
  DBMS_OUTPUT.PUT_LINE('Name    : ' || sname);
  DBMS_OUTPUT.PUT_LINE('Total   : ' || total);
  DBMS_OUTPUT.PUT_LINE('Percent : ' || perc);
  DBMS_OUTPUT.PUT_LINE('Class   : ' || class);
END;
/

```

(26) Generate a PL/SQL block to print 1 squared, 3 squared, 5 squared ... n using while loop.

```

DECLARE
  n NUMBER := &n;
  i NUMBER := 1;
BEGIN
  WHILE i <= n LOOP

```

```

        DBMS_OUTPUT.PUT_LINE(i*i);
        i := i + 2;
    END LOOP;
END;
/

```

(27) Generate a PL/SQL block to print the sum of 1, 4, 7 ... n value.

```

DECLARE
    n NUMBER := &n;
    i NUMBER := 1;
    s NUMBER := 0;
BEGIN
    WHILE i <= n LOOP
        s := s + i;
        i := i + 3;
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('Sum = ' || s);
END;
/

```

(28) Generate a PL/SQL block to print the 0 1 1 2 3 5 8 ... n series.

```

DECLARE
    n NUMBER := &n;    -- number of terms to print
    a NUMBER := 0;
    b NUMBER := 1;
    c NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE(a);
    DBMS_OUTPUT.PUT_LINE(b);
    FOR i IN 3..n LOOP
        c := a + b;
        DBMS_OUTPUT.PUT_LINE(c);
        a := b;
        b := c;
    END LOOP;
END;
/

```

(29) Generate a PL/SQL block to enter any number and print its factorial value.

```

DECLARE
    n NUMBER := &n;
    fact NUMBER := 1;
BEGIN
    FOR i IN 1..n LOOP
        fact := fact * i;
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('Factorial of ' || n || ' = ' || fact);
END;
/

```

(30) Write a PL/SQL program to check whether a given number is positive, negative or zero.

```

DECLARE
    n NUMBER := &n;
BEGIN
    IF n > 0 THEN
        DBMS_OUTPUT.PUT_LINE(n || ' is Positive');
    ELSIF n < 0 THEN

```

```

        DBMS_OUTPUT.PUT_LINE(n || ' is Negative');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Number is Zero');
    END IF;
END;
/

```

(31) Write a PL/SQL program to check whether a given character is letter or digit.

```

DECLARE
    ch CHAR(1) := '&ch';
BEGIN
    IF ch BETWEEN '0' AND '9' THEN
        DBMS_OUTPUT.PUT_LINE(ch || ' is a Digit');
    ELSIF (ch BETWEEN 'a' AND 'z') OR (ch BETWEEN 'A' AND 'Z') THEN
        DBMS_OUTPUT.PUT_LINE(ch || ' is a Letter');
    ELSE
        DBMS_OUTPUT.PUT_LINE(ch || ' is neither Letter nor Digit');
    END IF;
END;
/

```

(32) Generate a PL/SQL block to print the prime numbers between 1 to 50.

```

DECLARE
    isPrime BOOLEAN;
BEGIN
    FOR n IN 2..50 LOOP
        isPrime := TRUE;
        FOR i IN 2..TRUNC(SQRT(n)) LOOP
            IF MOD(n,i) = 0 THEN
                isPrime := FALSE;
                EXIT;
            END IF;
        END LOOP;
        IF isPrime THEN
            DBMS_OUTPUT.PUT_LINE(n);
        END IF;
    END LOOP;
END;
/

```

(33) Generate a PL/SQL block to display all the details of an employee, whose ID is 45.

```

DECLARE
    v_emp employees%ROWTYPE;
BEGIN
    SELECT * INTO v_emp FROM employees WHERE employee_id = 45;

    DBMS_OUTPUT.PUT_LINE('ID      : ' || v_emp.employee_id);
    DBMS_OUTPUT.PUT_LINE('Name    : ' || v_emp.first_name || ' ' || v_emp.last_name);
    DBMS_OUTPUT.PUT_LINE('Salary : ' || v_emp.salary);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employee found with ID 45');
END;
/

```

(34) Write a PL/SQL block that inserts a new employee into the employees table only if they are not already present.

```

DECLARE
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count FROM employees WHERE employee_id = &empid;

    IF v_count = 0 THEN
        INSERT INTO employees(employee_id, first_name, last_name, salary)
        VALUES (&empid, '&fname', '&lname', &sal);
        DBMS_OUTPUT.PUT_LINE('Employee Inserted');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Employee already exists');
    END IF;
END;
/

```

(35) Create the following table and write down a PL/SQL procedure to display records using an explicit cursor. Table: Student - Id (primary key), fname, lname, city

```

CREATE TABLE Student (
    Id      NUMBER PRIMARY KEY,
    fname   VARCHAR2(20),
    lname   VARCHAR2(20),
    city    VARCHAR2(20)
);

DECLARE
    CURSOR c_student IS SELECT * FROM Student;
    v_student c_student%ROWTYPE;
BEGIN
    OPEN c_student;
    LOOP
        FETCH c_student INTO v_student;
        EXIT WHEN c_student%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(v_student.Id || ' ' || v_student.fname || ' ' ||
                             v_student.lname || ' ' || v_student.city);
    END LOOP;
    CLOSE c_student;
END;
/

```

(36) Demonstrate the use of explicit cursor with ISOPEN, NOTFOUND attributes.

```

DECLARE
    CURSOR c_emp IS SELECT employee_id, first_name FROM employees;
    v_emp c_emp%ROWTYPE;
BEGIN
    IF NOT c_emp%ISOPEN THEN
        OPEN c_emp;
    END IF;

    LOOP
        FETCH c_emp INTO v_emp;
        IF c_emp%NOTFOUND THEN
            EXIT;
        END IF;
        DBMS_OUTPUT.PUT_LINE(v_emp.employee_id || ' - ' || v_emp.first_name);
    END LOOP;

    CLOSE c_emp;
END;
/

```

(37) Generate a Cursor to select the five highest paid employees from the employee table.

Note: The ROWNUM sub-query form above works on Oracle 10g/11g. On Oracle 12c and later you may instead write "ORDER BY salary DESC FETCH FIRST 5 ROWS ONLY".

```
DECLARE
  CURSOR c_top5 IS
    SELECT employee_id, first_name, salary FROM (
      SELECT employee_id, first_name, salary
      FROM employees
      ORDER BY salary DESC
    ) WHERE ROWNUM <= 5;
BEGIN
  FOR emp_rec IN c_top5 LOOP
    DBMS_OUTPUT.PUT_LINE(emp_rec.employee_id || ' ' ||
      emp_rec.first_name || ' ' || emp_rec.salary);
  END LOOP;
END;
/
```

(38) Write a PL/SQL block that uses an explicit cursor to fetch employee names and salaries.

```
DECLARE
  CURSOR c_emp IS SELECT first_name, salary FROM employees;
BEGIN
  FOR rec IN c_emp LOOP
    DBMS_OUTPUT.PUT_LINE(rec.first_name || ' - ' || rec.salary);
  END LOOP;
END;
/
```

(39) Write a PL/SQL block that attempts to divide two numbers and handles division by zero exceptions.

```
DECLARE
  a NUMBER := &a;
  b NUMBER := &b;
  result NUMBER;
BEGIN
  result := a / b;
  DBMS_OUTPUT.PUT_LINE('Result = ' || result);
EXCEPTION
  WHEN ZERO_DIVIDE THEN
    DBMS_OUTPUT.PUT_LINE('Error: Division by zero is not allowed');
END;
/
```

(40) Write a PL/SQL block that retrieves all the available employees in the table.

```
DECLARE
  CURSOR c_emp IS SELECT * FROM employees;
BEGIN
  FOR rec IN c_emp LOOP
    DBMS_OUTPUT.PUT_LINE(rec.employee_id || ' ' || rec.first_name || ' ' || rec.salary);
  END LOOP;
END;
/
```

(41) Write a recursive procedure that prints numbers in descending order from a given number.

```
CREATE OR REPLACE PROCEDURE print_descending(n NUMBER) IS
BEGIN
    IF n >= 1 THEN
        DBMS_OUTPUT.PUT_LINE(n);
        print_descending(n - 1);
    END IF;
END;
/

BEGIN
    print_descending(10);
END;
/
```

(42) Write a procedure to delete records from the employees table based on a given department ID.

```
CREATE OR REPLACE PROCEDURE delete_by_dept(p_deptid NUMBER) IS
BEGIN
    DELETE FROM employees WHERE department_id = p_deptid;

    IF SQL%ROWCOUNT = 0 THEN
        DBMS_OUTPUT.PUT_LINE('No records found for Department ' || p_deptid);
    ELSE
        DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' record(s) deleted');
    END IF;
    COMMIT;
END;
/

BEGIN
    delete_by_dept(&deptid);
END;
/
```

(43) Write a PL/SQL block that checks if a specific employee exists in the employees table.

```
DECLARE
    v_count NUMBER;
    v_empid NUMBER := &empid;
BEGIN
    SELECT COUNT(*) INTO v_count FROM employees WHERE employee_id = v_empid;

    IF v_count > 0 THEN
        DBMS_OUTPUT.PUT_LINE('Employee Exists');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Employee Does Not Exist');
    END IF;
END;
/
```

(44) Write a PL/SQL block for displaying sid, sname, DOB, address, city from student table using a procedure with a call block.

```
CREATE OR REPLACE PROCEDURE show_student(p_sid NUMBER) IS
    v_sname VARCHAR2(30);
    v_dob DATE;
```

```

v_address VARCHAR2(50);
v_city    VARCHAR2(20);
BEGIN
    SELECT sname, DOB, address, city
    INTO v_sname, v_dob, v_address, v_city
    FROM student
    WHERE sid = p_sid;

    DBMS_OUTPUT.PUT_LINE('Sid      : ' || p_sid);
    DBMS_OUTPUT.PUT_LINE('Sname    : ' || v_sname);
    DBMS_OUTPUT.PUT_LINE('DOB      : ' || v_dob);
    DBMS_OUTPUT.PUT_LINE('Address : ' || v_address);
    DBMS_OUTPUT.PUT_LINE('City     : ' || v_city);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Student not found');
END;
/

BEGIN
    show_student(&sid);    -- call block
END;
/

```

(45) Write a PL/SQL code for a stored procedure to change a student's name to capital letters by passing the Roll No and raise an appropriate error if the student does not exist in the student table.

```

CREATE OR REPLACE PROCEDURE change_name_caps(p_rollno NUMBER) IS
    v_count NUMBER;
    e_no_student EXCEPTION;
BEGIN
    SELECT COUNT(*) INTO v_count FROM student WHERE rollno = p_rollno;

    IF v_count = 0 THEN
        RAISE e_no_student;
    ELSE
        UPDATE student
        SET sname = UPPER(sname)
        WHERE rollno = p_rollno;
        DBMS_OUTPUT.PUT_LINE('Name updated to uppercase');
        COMMIT;
    END IF;
EXCEPTION
    WHEN e_no_student THEN
        RAISE_APPLICATION_ERROR(-20001,
            'Student with Roll No ' || p_rollno || ' does not exist');
END;
/

BEGIN
    change_name_caps(&rollno);
END;
/

```

(46) Create a procedure that counts the number of employees in a particular department.

```

CREATE OR REPLACE PROCEDURE count_emp_dept(p_deptid NUMBER) IS
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count FROM employees WHERE department_id = p_deptid;

```



```

        DBMS_OUTPUT.PUT_LINE('Employees in Department ' || p_deptid || ' = ' || v_count);
END;
/

BEGIN
    count_emp_dept(&deptid);
END;
/

```

(47) Create a procedure to add a record into table student. Table: Student - Sid, sname, age, class

```

CREATE OR REPLACE PROCEDURE add_student(
    p_sid NUMBER, p_sname VARCHAR2, p_age NUMBER, p_class VARCHAR2
) IS
BEGIN
    INSERT INTO student(sid, sname, age, class)
    VALUES (p_sid, p_sname, p_age, p_class);
    DBMS_OUTPUT.PUT_LINE('Student Record Added');
    COMMIT;
END;
/

BEGIN
    add_student(&sid, '&sname', &age, '&class');
END;
/

```

(48) Create a package that contains two procedures - one to add a record and one to delete a record. Choose a suitable table.

```

CREATE OR REPLACE PACKAGE student_pkg IS
    PROCEDURE add_student(p_sid NUMBER, p_sname VARCHAR2, p_age NUMBER, p_class VARCHAR2);
    PROCEDURE delete_student(p_sid NUMBER);
END student_pkg;
/

CREATE OR REPLACE PACKAGE BODY student_pkg IS
    PROCEDURE add_student(p_sid NUMBER, p_sname VARCHAR2, p_age NUMBER, p_class VARCHAR2) IS
    BEGIN
        INSERT INTO student(sid, sname, age, class)
        VALUES (p_sid, p_sname, p_age, p_class);
        DBMS_OUTPUT.PUT_LINE('Record Added');
        COMMIT;
    END add_student;

    PROCEDURE delete_student(p_sid NUMBER) IS
    BEGIN
        DELETE FROM student WHERE sid = p_sid;
        DBMS_OUTPUT.PUT_LINE('Record Deleted');
        COMMIT;
    END delete_student;
END student_pkg;
/

BEGIN
    student_pkg.add_student(1, 'Rahul', 20, 'BCA');
    student_pkg.delete_student(1);
END;
/

```

(49) Write a PL/SQL code to find out the square of a number using the INOUT parameter of a procedure.

```
CREATE OR REPLACE PROCEDURE find_square(p_num IN OUT NUMBER) IS
BEGIN
    p_num := p_num * p_num;
END;
/

DECLARE
    v_num NUMBER := &n;
BEGIN
    find_square(v_num);
    DBMS_OUTPUT.PUT_LINE('Square = ' || v_num);
END;
/
```

(50) Write a PL/SQL code to find out the minimum number using IN and OUT parameters of a procedure.

```
CREATE OR REPLACE PROCEDURE find_min(p_a IN NUMBER, p_b IN NUMBER, p_min OUT NUMBER) IS
BEGIN
    IF p_a < p_b THEN
        p_min := p_a;
    ELSE
        p_min := p_b;
    END IF;
END;
/

DECLARE
    v_min NUMBER;
BEGIN
    find_min(&a, &b, v_min);
    DBMS_OUTPUT.PUT_LINE('Minimum = ' || v_min);
END;
/
```

(51) Write a PL/SQL code for a trigger that performs a delete operation on the Main table and the deleted record should be inserted into the Backup table.

```
CREATE TABLE Main_Table (id NUMBER, name VARCHAR2(20), salary NUMBER);
CREATE TABLE Backup_Table (id NUMBER, name VARCHAR2(20), salary NUMBER, deleted_on DATE);

CREATE OR REPLACE TRIGGER trg_backup_on_delete
BEFORE DELETE ON Main_Table
FOR EACH ROW
BEGIN
    INSERT INTO Backup_Table(id, name, salary, deleted_on)
    VALUES (:OLD.id, :OLD.name, :OLD.salary, SYSDATE);
END;
/
```

(52) Write a PL/SQL code that, when an insert or update is performed with salary less than 5000, raises an error using a trigger.

```
CREATE OR REPLACE TRIGGER trg_check_salary
BEFORE INSERT OR UPDATE ON employees
FOR EACH ROW
BEGIN
    IF :NEW.salary < 5000 THEN
```

```

        RAISE_APPLICATION_ERROR(-20002, 'Salary cannot be less than 5000');
    END IF;
END;
/

```

(53) Write a database trigger which will not allow any transaction after office hours (after 7:00 PM) and on Sunday.

```

CREATE OR REPLACE TRIGGER trg_restrict_time
BEFORE INSERT OR UPDATE OR DELETE ON employees
BEGIN
    IF TO_CHAR(SYSDATE, 'DY') = 'SUN' OR TO_CHAR(SYSDATE, 'HH24') >= '19' THEN
        RAISE_APPLICATION_ERROR(-20003,
            'Transactions are not allowed after 7 PM or on Sunday');
    END IF;
END;
/

```

(54) Write a trigger to record any changes made to the salary column in the salary_history table.

```

CREATE TABLE salary_history (
    emp_id      NUMBER,
    old_salary  NUMBER,
    new_salary  NUMBER,
    changed_on  DATE
);

CREATE OR REPLACE TRIGGER trg_salary_history
AFTER UPDATE OF salary ON employees
FOR EACH ROW
BEGIN
    INSERT INTO salary_history(emp_id, old_salary, new_salary, changed_on)
    VALUES (:OLD.employee_id, :OLD.salary, :NEW.salary, SYSDATE);
END;
/

```

(55) Write an update trigger on the client master table. The system keeps a record of changes being updated. The old values of the update should be added to the Temp table. Table: Temp - C_no, name, operation, o_date, uid

```

CREATE TABLE Temp (
    C_no      NUMBER,
    name      VARCHAR2(30),
    operation  VARCHAR2(10),
    o_date    DATE,
    uid       VARCHAR2(20)
);

CREATE OR REPLACE TRIGGER trg_client_update
BEFORE UPDATE ON client_master
FOR EACH ROW
BEGIN
    INSERT INTO Temp(C_no, name, operation, o_date, uid)
    VALUES (:OLD.c_no, :OLD.name, 'UPDATE', SYSDATE, USER);
END;
/

```

(56) Write a code in PL/SQL to create a trigger that prevents updates on a certain column during specific hours of the day.

```
CREATE OR REPLACE TRIGGER trg_restrict_column_update
BEFORE UPDATE OF salary ON employees
FOR EACH ROW
BEGIN
    IF TO_CHAR(SYSDATE, 'HH24') BETWEEN '09' AND '18' THEN
        RAISE_APPLICATION_ERROR(-20004,
            'Salary cannot be updated between 9 AM and 6 PM');
    END IF;
END;
/
```

(57) Create a trigger that automatically records any insertions into the audit_log table whenever a new employee is added.

```
CREATE TABLE audit_log (
    emp_id      NUMBER,
    action      VARCHAR2(20),
    action_date DATE
);

CREATE OR REPLACE TRIGGER trg_audit_insert
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
    INSERT INTO audit_log(emp_id, action, action_date)
    VALUES (:NEW.employee_id, 'INSERT', SYSDATE);
END;
/
```

(58) Write a code in PL/SQL to create a trigger that checks for duplicate values in a specific column and raises an exception if found.

```
CREATE OR REPLACE TRIGGER trg_check_duplicate
BEFORE INSERT ON employees
FOR EACH ROW
DECLARE
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM employees
    WHERE email = :NEW.email;

    IF v_count > 0 THEN
        RAISE_APPLICATION_ERROR(-20005, 'Duplicate email is not allowed');
    END IF;
END;
/
```

(59) Write a PL/SQL code to create a package that includes a procedure to calculate the factorial of a number and a function to check if a number is prime.

```
CREATE OR REPLACE PACKAGE math_pkg IS
    PROCEDURE find_factorial(p_num IN NUMBER);
    FUNCTION is_prime(p_num IN NUMBER) RETURN BOOLEAN;
END math_pkg;
/

CREATE OR REPLACE PACKAGE BODY math_pkg IS
```

```

PROCEDURE find_factorial(p_num IN NUMBER) IS
    v_fact NUMBER := 1;
BEGIN
    FOR i IN 1..p_num LOOP
        v_fact := v_fact * i;
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('Factorial of ' || p_num || ' = ' || v_fact);
END find_factorial;

FUNCTION is_prime(p_num IN NUMBER) RETURN BOOLEAN IS
BEGIN
    IF p_num < 2 THEN
        RETURN FALSE;
    END IF;
    FOR i IN 2..TRUNC(SQRT(p_num)) LOOP
        IF MOD(p_num, i) = 0 THEN
            RETURN FALSE;
        END IF;
    END LOOP;
    RETURN TRUE;
END is_prime;
END math_pkg;
/

DECLARE
    v_result BOOLEAN;
BEGIN
    math_pkg.find_factorial(5);
    v_result := math_pkg.is_prime(7);
    IF v_result THEN
        DBMS_OUTPUT.PUT_LINE('7 is Prime');
    ELSE
        DBMS_OUTPUT.PUT_LINE('7 is Not Prime');
    END IF;
END;
/

```

(60) Create a package to handle employee salary operations: one procedure to increase salary, and one to decrease salary.

Note: The original question text was cut off after "one procedure to increase salary," so a matching decrease_salary procedure has been added to complete the package.

```

CREATE OR REPLACE PACKAGE salary_pkg IS
    PROCEDURE increase_salary(p_empid NUMBER, p_amount NUMBER);
    PROCEDURE decrease_salary(p_empid NUMBER, p_amount NUMBER);
END salary_pkg;
/

CREATE OR REPLACE PACKAGE BODY salary_pkg IS
    PROCEDURE increase_salary(p_empid NUMBER, p_amount NUMBER) IS
    BEGIN
        UPDATE employees
        SET salary = salary + p_amount
        WHERE employee_id = p_empid;
        DBMS_OUTPUT.PUT_LINE('Salary Increased');
        COMMIT;
    END increase_salary;

    PROCEDURE decrease_salary(p_empid NUMBER, p_amount NUMBER) IS
    BEGIN
        UPDATE employees
        SET salary = salary - p_amount
        WHERE employee_id = p_empid;
    END decrease_salary;
END salary_pkg;
/

```

```
        DBMS_OUTPUT.PUT_LINE('Salary Decreased');
    COMMIT;
END decrease_salary;
END salary_pkg;
/

BEGIN
    salary_pkg.increase_salary(101, 2000);
    salary_pkg.decrease_salary(102, 1000);
END;
/
```